오픈소스 반도체 설계 및 제작 종합설계 5주차 학습노트

이수빈

Gemini와의 대화로 만든 Space Invaders 게임의 베릴로그 RTL을 시뮬레이션에서 확인하고, FPGA 보드에 올려 실제 회로로 동작시키는 과정을 정리한다.

1. 진행 내용

1.1 시뮬레이션

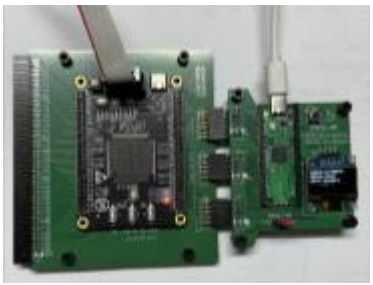
게임 RTL은 Verilator로 C++ 모델로 바꾸고, SystemC 테스트벤치와 SDL 화면 모델을 붙여 PC에서 돌렸다. Q.1과 Q.4는 그래픽 장치를 다루는 방식이 달랐다. Q.1은 외부에서 좌표를 입력으로 받아 색을 계산해 돌려주는 구조였고, Q.4는 RTL이 스캔 카운터로 128x64 화면을 직접 훑으며 좌표와 값을 출력하는 구조였다. 한 화면은 8,192 클럭이다. 소프트웨어(아두이노)에서는 스프라이트를 순서대로 그리지만 RTL에서는 비교 회로가 병렬로 동작해서, 외계인 25마리를 판정하는 비교기가 25번 생기는 것을 확인했다.

1.2 키보드와 RTL의 시간 척도

Q.11에서는 방향키를 짧게 눌러도 캐논이 벽에 붙었다. RTL은 버튼이 눌려 있는 동안 매 클럭 한 칸씩 움직이는데, 테스트벤치는 키보드를 훨씬 느리게 읽었기 때문이다. Q.13에서는 화면의 마지막 점에서만 버튼을 한 번 읽도록 RTL을 고쳐서 해결했다. Q.13에는 RTL이 다섯 버전 들어 있었고, 여러 버전을 직접 돌려 보니 마지막 버전(4)에서 캐논과 총알이 모두 정상으로 동작했다. Q.14의 RTL을 보면 카운터 하나의 하위 7비트를 x 좌표, 상위 6비트를 y 좌표로 쓰고, 외계인 25마리는 좌표를 각각 저장하지 않고 25비트 레지스터로 생사만 구분하고 있었다.

1.3 FPGA 보드

Cyclone IV E(EP4CE6) FPGA 보드와 Pico 보드를 사용했다.



Quartus로 컴파일하고 openFPGALoader로 올렸는데, 처음에는 IDCODE를 알 수 없다는 오류가 나서 최신 소스를 직접 빌드했다. Pico가 100 kHz 클럭을 FPGA에 주고 PC에서는 키보드 상태만 보내는 프로토타이핑으로 Q.14를 돌렸고, OLED에서 START 화면(체커보드)을 거쳐 게임 화면까지 확인했다.



처음에는 PC 앱에서 스페이스가 먹지 않았다. 시리얼 포트로 바이트를 직접 보내 보니 정상이어서 배선 문제가 아님을 알았고, PC 앱이 가상 클럭마다 같은 값을 계속 보내 시리얼 쓰기가 막힌 것이 원인으로 보여 값이 바뀔 때만 보내도록 고쳤다. 고친 뒤 화살표와 발사 모두 정상 동작했다.

2. 결과

에뮬레이션은 시뮬레이션에서 쓰던 SystemC 테스트벤치로 FPGA에 올린 회로를 시험하는 것이고, 프로토타이핑은 FPGA에 올린 게임을 Pico가 만든 클럭으로 실제처럼 돌려 보는 것이다. 에뮬레이션용 FPGA 파일에는 게임 회로를 감싸 Pico와 입출력을 주고받는 래퍼(wrapper)라는 껍질 회로가 함께 들어간다.

	Q.13 프로토타이핑	Q.14 프로토타이핑	Q.14 에뮬레이션 (래퍼wrapper 포함)
논리소자	870	817	683
레지스터	150	111	122
핀	21	8	24

Q.13에서 Q.14로 가면서 핀은 21개에서 8개로, 레지스터는 150개에서 111개로 줄었고, 논리 소자는 870개에서 817개로 줄었다. Q.14 에뮬레이션용은 래퍼가 더해졌는데도 논리 소자가 683개로 오히려 적었다. 핀은 래퍼의 입출력이 고정이어서 24개였다.

세 가지 방식의 차이는 다음과 같이 정리했다.

	시뮬레이션	에뮬레이션	프로토타이핑
DUT	Verilator가 만든 C++ 모델	FPGA 회로	FPGA 회로
클럭	SystemC 커널	테스트벤치가 클럭마다 USB로 진행	Pico가 PWM 100 kHz로 생성
테스트벤치	SystemC	시뮬레이션과 같은 SystemC	키보드 입력 프로그램
화면	PC의 SDL 창	SDL 창과 OLED	OLED

3. 고찰

3.1 언어로 하는 시뮬레이션(Language Simulation)과 하드웨어는 다르다

소프트웨어 언어로 하드웨어를 기술한다는 것에는 처음부터 한계가 있다고 생각한다. 베릴로그의 문법이 맞고 논리가 맞아도, 시뮬레이터는 코드를 정해진 순서로 계산하는 프로그램이고 실제 회로는 모든 게이트와 플립플롭이 동시에 동작한다. 연속 대입이 많이 섞여 있으면 이 차이가 드러난다. 연속 대입은 클럭 에지에서만 값이 필요한 신호에는 의미가 없고, 같은 코드가 시뮬레이션에서는 맞게 보여도 합성하면 다르게 동작할 수 있다. 그래서 값이 저장되는 곳(클럭에 맞춘 always)과 바로 계산되는 곳(조합 논리)을 나누어 쓰는 것이 중요하다고 생각한다. Q.13에서 이런 예를 직접 보았다. 버전 3은 current_state를 상태 레지스터 블록과 게임 로직의 두 곳에서 바꾸고 있어서, Quartus로 컴파일하면 current_state의 여러 드라이버를 해결할 수 없다는 오류(Error 10028)가 나고 합성이 끝나지 않았다. 버전 0도 같은 구조였지만 시뮬레이션에서는 돌아갔다. 시뮬레이션만 보아서는 드러나지 않는 문제였다.

3.2 시간 척도를 클럭 수로 맞추는 일

Q.11의 문제는 결국 시간 척도의 문제였다. RTL의 한 클럭과 사람이 키를 누르는 시간 사이에는 수만 번의 클럭이 지나가기 때문에, 한 번의 입력이 수만 번의 이동으로 불어났다. 해결은 화면이 끝나는 순간에만 입력을 읽는 것이었다. 한 화면은 8,192 클럭이고, 캐논은 화면마다 한 칸, 외계인은 네 화면마다 한 칸 움직인다. 이렇게 모든 움직임을 클럭 수로 정해 두면 클럭이 10 MHz이든 보드의 100 kHz이든 게임의 규칙은 같고 실제 속도만 달라진다.

3.3 칩의 제약이 구조를 정한다

Q.13과 Q.14의 입출력이 다른 이유가 가장 기억에 남는다. 칩의 핀 28개 중 전원과 접지가 16개여서 신호 핀은 12개뿐이다. Q.13의 입출력은 21개여서 들어갈 수 없었고, 좌표 13비트를 내보내는 대신 화면을 순서대로 훑으며 화소 하나만 내보내는 구조로 바뀌었다. 좌표를 만드는 일은 받는 쪽인 Pico 펌웨어로 넘어갔다. 그래서 화면 한 장이 끝났다는 신호(v_sync)가 필요했고, 샘플링 시점을 맞추기 위해 클럭을 반전해 내보내는 clk_o도 필요했다. 이번 경우에는 인터페이스가 패드와 핀 수 같은 물리적 조건에서 먼저 정해졌고, 그 위에 RTL 구조가 맞춰졌다. Quartus 결과에서 핀은 21개에서 8개로 줄었지만 논리 소자는 870개에서 817개로 거의 그대로였다. 게임 로직 자체는 그대로이고 바깥과 주고받는 방식만 바뀐 것이다.

3.4 같은 테스트벤치로 확인한다는 것

시뮬레이션, 에뮬레이션, 프로토타이핑은 같은 게임 RTL을 서로 다른 곳에서 돌려 본 것이다. 시뮬레이션에서는 RTL이 Verilator가 만든 C++ 모델이 되어 PC 안에서 돌고, 에뮬레이션과 프로토타이핑에서는 FPGA에 합성된 회로로 돈다. 시뮬레이션에서 본 동작이 실제 회로에서도 같은지는 같은 테스트벤치로 두 결과를 나란히 놓고 비교해야 알 수 있다고 생각한다. 내가 직접 돌려 본 프로토타이핑은 이 비교가 아니라, 키보드 입력만 보내고 화면은 OLED로 보면서 게임이 실제로 동작하는지를 보는 것이었다.

3.5 코드 밖의 조건

이번에 가장 오래 걸린 일은 RTL이 아니라 코드 밖에서 생겼다. FPGA 로더가 칩의 ID를 모르는 문제, USB를 WSL로 넘겨 주는 과정, Pico를 업로드 모드로 바꾸면 장치가 Windows 쪽으로 가는 문제가 그랬다. PC 앱이 시리얼 쓰기에서 멈춘 것도 같은 종류였다. 이런 프로그램은 상대의 응답을 기다리는 구조라서 행(hang)이 걸리면 창을 닫아도 꺼지지 않았다. 시뮬레이션에서는 시뮬레이터가 모든 것을 같은 프로그램 안에서 돌리지만, 보드에서는 PC, USB, Pico, FPGA가 각자의 속도로 동작하기 때문에 한 곳에서 기다리면 전체가 멈춘다. RTL이 맞아도 연결과 환경이 틀리면 게임이 돌아가지 않는다는 것을 겪으며 하드웨어 검증의 범위가 코드보다 넓다는 것을 느꼈다.

4. 과제를 수행하며

이번 과제를 하면서 처음에는 게임이 돌아가기만 하면 된다고 생각했지만, 돌아가는 것과

이유를 아는 것은 다르다는 것을 여러 번 느꼈다. 캐논이 벽에 붙는 문제도 처음에는 키보드가 이상한 것 같았지만, 시간 척도라는 한 가지 원인으로 설명되었다.

궁금한 점도 계속 생겼다. 첫째는 버튼의 채터링이다. 실제 보드의 버튼은 접점이 닿는 순간 신호가 0과 1 사이를 여러 번 오갈 텐데, 지금의 RTL은 발사 버튼의 상승 에지를 잡아 한 발을 쏜다. 상승 에지가 여러 번 잡히면 한 번 눌렀는데 여러 발이 나갈 수 있다. 버튼을 화면 한 장에 한 번만 읽는 지금 구조에서 떨림이 어디까지 걸러지는지, 엣지 검출만으로 충분한지 알고 싶다. 버튼 값이 몇 클럭 동안 같게 유지될 때만 인정하는 회로가 필요한지도 궁금하다.

둘째는 Q.13과 Q.14의 차이를 어디서 맞추는가이다. Q.13은 RTL이 좌표를 내보내고 화면 모델이 그것을 그대로 읽는다. Q.14는 화소만 내보내므로 화면 모델이 clk_o가 올라올 때마다 화소를 세고, v_sync에서 0으로 되돌려 좌표를 만든다. 이 일은 Pico 펌웨어가 하는 일과 같았다. 시뮬레이션에서는 화면 모델이, 보드에서는 Pico가 같은 역할을 하는 것이다. 같은 테스트벤치를 쓴다고 해도 RTL의 인터페이스가 바뀌면 화면 모델은 따라 바뀌어야 한다.

셋째는 에뮬레이션용 래퍼에서 논리 소자가 줄어든 이유이다. 래퍼가 더해졌는데도 논리 소자가 프로토타이핑용(817개)보다 적은 683개였다. 이유를 Quartus의 합성 리포트에서 제거된 레지스터 목록으로 직접 확인해 보고 싶다.